

The Host Is Part of the Block: A Goal-Pursuit Ladder at Two Scales and the Composition-Matching Law

Franci Penov
kortexa.ai
francip@kortexa.ai

2026-08-11

Abstract

The cadence notes [1, 2] left a goal living in a conditioning channel and a creature-scale loop that holds it; this note makes the goal DO something. A registered ladder (G0-G3 plus a temporal-pressure rung) puts a frozen host on a chained-clue errand — look, open, submit, under a tick budget, with every episode’s actions and observations recorded as first-class transcripts — at 230M with the validated activation-packet channel and at 27B with the validated weight-block channel. Three findings. First, the scale answer: zero-shot, BOTH hosts are below the task (0.00 with the goal in text); trained by 384 steps of oracle cloning, both are perfect when the goal rides in text (1.00 at every seed, both venues); trained with the goal in the channel, both carry a partial want (0.70/0.80/1.00 at 230M, 0.60/0.80/0.70 at 27B) that directs behavior without surviving symbol-level conflict — the interesting gap is channel-vs-text, not small-vs-large. Second, the environment fought the preregistration three times — a reveal-order leak, a mis-specified control floor, a Markovian degeneracy — and the registered controls caught all three; the diagnosis runs are kept. Third, and the note’s title: a conditioning artifact works exactly on the host composition it was trained under. The same emitter that passes its gate in-harness passes through the production server’s cached-generate delivery path on a matched host (0.60 vs floors 0.20/0.00) and collapses to chance when one more block arms (0.20), because the harness composes blocks by addition and the server by factor averaging — a cross-term operator mismatch independently documented in the serving stack’s own audit. The law held four times in one arc (two GT rungs, two canaries), and its constructive corner stands: packets retrained under the moving host keep their write-once regime at 0.97–1.00, and the trained errand policy delivered through the real serving path reaches 0.40 against a 1.00 harness ceiling and a 0.00 serve floor — the transfer gap is now a measured number, not a fear. (Addendum, 2026-08-15: the registered re-runs turn both of these numbers over. Training the policy under the temporal-armed host inverts the GT table exactly — temporal 1.00, the text arms 0.00 — and training it under the host the SERVER actually presents, which is bare because an unfed block contributes nothing, closes the 0.40 serve gap to 1.00. The law holds in both directions, and the serve wrapper costs nothing. The pressure rung the note leaves open also ran: under a hidden variable budget with noisy observations, the temporal block gives the frozen 230M the same wall-timed submission behavior a countdown sentence gives it, with frozen-time and shuffled-time controls both at the no-signal baseline.)

1 From a want that persists to a want that acts

Paper 12 measured a goal packet that endures; paper 13 closed its re-encoding gap. The registered goal-pursuit line asks the next question: can the want DRIVE something — a multi-step errand

venue	trained text	goal block	block done-choice	no-goal floor
230M (activation packet)	1.00/1.00/1.00	0.70/0.80/1.00	0.70/0.89/1.00	0.60/0.80/0.70
27B (weight block)	1.00/1.00/1.00	0.60/0.80/0.70	0.60/0.80/0.70	0.50/0.50/0.80

Table 1: G1 at both venues, per seed. Both channels carry a partial want; text carries a whole one; the floors are the trained goal-free bridges whose submit is a coin flip.

through tools — and where does the want have to live for that to work? The environment is deliberately small and fully machine-validated: eight places, eight items, three-hop clue chains (a note names the next place, the last place holds the target beside a generator-drawn distractor, submit ends the episode), a tick budget of eight, greedy decode, and a transcript for every episode. The distractor makes the final submit a forced choice only the goal can decide; the oracle needs four ticks. Venues: the 230M creature stack (dream4 armed, the activation-packet channel the C-line validated) and the bare 27B column (the weight channel the multi-sensor line validated) — per-venue channels, registered as such, because the claim is about WHERE the want lives, not one mechanism.

2 The environment fought back, and the controls won

Three times the preregistered design was wrong, and three times its own control cells said so before any claim shipped. The first G1 run scored 1.00 everywhere INCLUDING the goal-free floor whose submit is a coin flip by construction — the reveal listed the target first, and every trained arm had learned “submit the first-listed item.” The floor cell and the shuffled control caught it; the order is now randomized; the leak run is kept as the diagnosis. The shuffled check itself was then found mis-specified (registered against an untrained floor any trained policy beats) and re-registered against the trained goal-free floor. And G2’s truncation rung returned its registered reading (iii): each clue names the next move, so the errand is Markovian and last-observation context recovers full performance exactly — the rung cannot separate state carriers on this task shape, recorded as a task-design note. Its one live cell: a goal entering the packet once and surviving on-policy re-encoding lands at 0.60/0.50/0.60 — direction above chance, weaker than text.

3 The scale answer

Table 1 and Figure 1 (left) carry it. Zero-shot, the errand defeats both hosts — 0.00 success with the goal stated in text, at 230M and at 27B alike (the 27B’s transcripts do not even wander well). Trained by oracle cloning with the goal in text, both hosts are perfect: 1.00 at every seed, both venues. Trained with the goal ONLY in the channel, both venues land in the same band — 0.70/0.80/1.00 at 230M through the activation packet, 0.60/0.80/0.70 at 27B through the weight block — above their trained goal-free floors, below their text ceilings, and in packet-vs-context conflict cells the channel’s item is almost never what gets submitted. The channel carries direction, not symbols, at both scales; scale bought nothing on the channel; and the question the program inherits is not “is the model big enough” but “what does a channel have to be for a want to survive contact with language.”

venue	none	countdown	temporal block
230M	1.00/1.00/1.00	1.00/1.00/1.00	0.00/0.00/0.00
27B	1.00/1.00/1.00	1.00/1.00/1.00	0.00/1.00/0.00

Table 2: GT per seed: the text arms are ceilinged (the trained policy is oracle-optimal, so the tight budget exerts no pressure) and the temporal arm measures host mismatch, not time-feeling — the bridge never trained under the summed host.

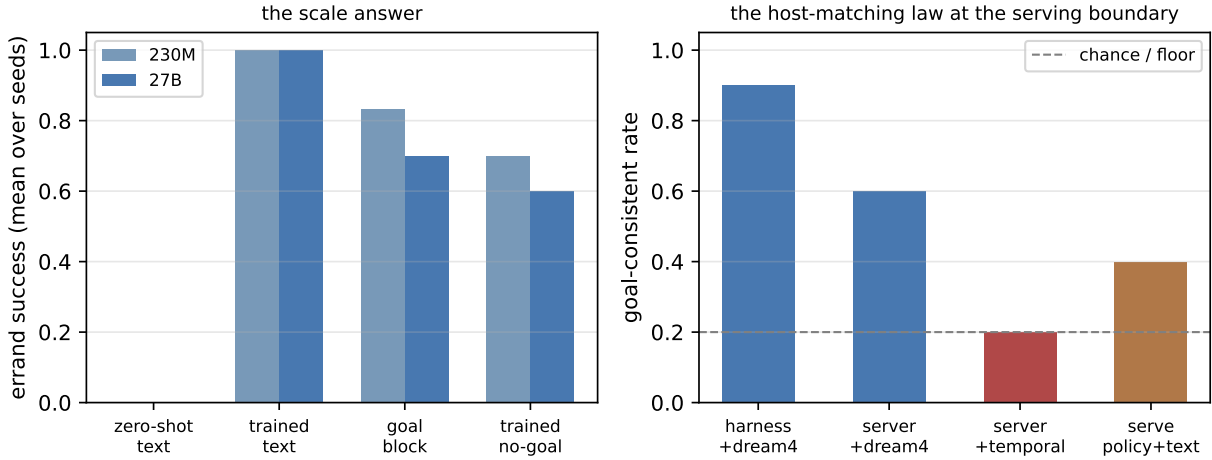


Figure 1: Left: the scale answer — zero-shot below task, trained-text perfect, channel partial, at both venues. Right: the law at the serving boundary — the same artifacts pass on matched hosts (harness+dream4, server+dream4), collapse under one extra block (server+temporal, the operator mismatch), and the trained policy delivered through the real serving path reaches 0.40 against a 1.00 harness ceiling.

4 The composition-matching law

Four independent cells, one law: *a conditioning artifact works on the host composition it was trained under, and on no other*. (1) GT at 230M: a policy bridge trained on the dream4-only host scores 1.00 — and 0.00 the moment the production temporal block’s delta is summed underneath it at evaluation (none/countdown/temporal = 1.00/1.00/1.00 / 1.00/1.00/1.00 / 0.00/0.00/0.00). (2) GT at 27B reproduces the shape (0.00/1.00/0.00 on the temporal arm). (3) The serving canary: the same emitter that passes its C1 gate in-harness passes through the production server’s cached-generate delivery path on a matched dream4-only host — true 0.60 against floors 0.20 and 0.00 — validating the delivery wiring end to end. (4) Arm one more block and the same canary collapses to chance (0.20), even for an emitter retrained under the moving temporal host; the residual mismatch is the OPERATOR: the harness composes blocks by flat-delta addition, the server by LoRA factor averaging with cross-terms — a bug the serving stack’s own audit had independently documented. Two investigation lines, one root cause; the fix pays both. The law’s constructive corner: the emitter retrained under the moving host keeps its write-once packets at 0.97–1.00 across sixteen ticks (1.00 early, 0.97 late), while its live re-encoding loop degrades (0.55/0.25) — the store architecture is robust where the self-loop is not.

5 The serving stake, measured

G3 ran the errand against an isolated instance of the production server — the real prompt wrapper, block composition, and cached-generate path, never the production port. Zero-shot serve floors: 0.00 with and without goal text; oracle 1.00 through the same plumbing. Then the trained policy bridge, exported as a packet store and delivered through the server’s own generation path: **policy+goal-text 0.40** — the first positive serve-side block result in the program — and policy without goal text 0.10, confirming the division of labor (the channel carries competence, the text carries the want). The serve wrapper’s cost against the 1.00 harness ceiling is now a number (0.60), with a registered follow-up: retrain the policy under the serve template — the same law, applied to the prompt regime.

6 What stands

A creature-scale stack now has the full chain measured: a want that persists (paper 12), re-writes itself (paper 13), directs a multi-step errand when trained (this note), and partially survives delivery through the real serving path — with the one blocking defect between here and the live creature located, named, and independently confirmed: the serving stack’s composition operator. The law this arc kept re-measuring belongs beside papers 7 and 9’s negatives: zero weight-space interference was never enough, and matching the host — its blocks, its operator, its template — is not a detail of deployment but part of the block’s identity. Every registration, amendment, diagnosis run, and full episode transcript is in the tracks’ PLAN files and the bundled artifacts.

7 Reproducibility

Twenty artifacts are bundled beside this note — the goal-pursuit ladder at both venues (including the kept leak-diagnosis run), the serve-composition emitter, both canaries, and the addendum’s four runs (the hostmatched re-run, its T=4 closure pair, the bare-host serve retrain, and the pressure rung with its two time controls); every number and the figure render from them alone. Harnesses: `goal_pursuit.py`, `goal_pursuit_27b.py`, `g3_serve.py`, `g3b_retrain.py` and `gtn_pressure.py` in `tracks/goal-pursuit/`; `own_cadence.py` and `c3b_canary.py` in `tracks/own-cadence/`; registrations and results, each dated before its run, in the tracks’ PLAN.md files.

Addendum (2026-08-15): the exact inversion

The registered follow-up ran: the same GT recipe at 230M with one change — the regime bridge’s training forwards run under the temporal-armed host, the tick sampled uniformly per example (the production-composition discipline the serving arc converged on). The outcome is the original run’s numbers with the arms swapped seat for seat (Table 3): the temporal arm scores 1.00/1.00/1.00 with mean done-tick 4.0 flat — the oracle path exactly — while none and countdown fall to 0.00/0.00/0.00 with ~50 invalid actions per cell, because now *they* are the mismatched compositions: their evaluation host is dream4-only and the bridge trained under dream4+temporal. Same arms, same chains, same budget; the only variable is which host the bridge trained under, and it fully determines the outcome in both directions. This is the law’s seventh sighting across the program and the first with a full symmetric swap. A same-day closure pair at T=4 — zero oracle slack — reproduces the swap seat for seat with freshly trained bridges: removing the last slack tick changes nothing, because the matched policy is oracle-exact and feels no pressure. It also retroactively

settles Table 2’s temporal column: 0.00 was never a time-feeling negative, only the law again. Artifacts `goal-pursuit-gt-230m-hostmatched.*` join the bundle.

And the time question itself, answered. GT could not test time-feeling because its trained policy is oracle-exact: done-tick 4.0 flat leaves nothing for urgency to buy. The registered pressure rung supplies something to buy — a HIDDEN budget drawn per chain from $\{4 \dots 8\}$ and never stated, plus observation noise ($\epsilon = 0.35$ per look at a clue) — so the optimal policy must retry smudged looks while the remaining ticks cover the path and submit the goal item the instant they do not. Every eval chain runs at every budget with its noise tape held fixed, which makes the comparison a within-chain identity test: the visible log is the same across budgets, so an arm with no time signal *cannot* behave differently, and does not. Per-arm host matching throughout, so no cell measures the law by accident.

With no time signal the model scores 0.94 and dies at the wall; with a countdown sentence, 0.98–1.00; with the temporal block and no time words anywhere in the context, **0.98–1.00**. Every failure in the grid sits at $T = 4$, and pooling those rows the decomposition is clean: temporal 0.97 against a shuffled-time control at 0.73 ($z = +2.53$), the shuffled control against a frozen-time control at 0.67 ($z = +0.56$), and the frozen control against no signal at all, 0.67 versus 0.70 ($z = -0.28$). A temporal block frozen at one timestamp buys nothing; one whose delta varies exactly as much but at random (t', T') buys nothing; only the delta that tracks the real remaining time helps — and it helps as much as the sentence does. The guess-rate says the same mechanically: at $T = 4$ the arms that submit early without opening the last place are precisely the two that know the time (temporal 0.20–0.30, countdown 0.20–0.30) against 0.00–0.20 for the three that do not. A weight-space block gives a frozen 230M the wall-timed behavior a countdown gives it, without symbols. The claim is bounded to the pressure rows: at $T \geq 5$ every arm is at 1.00. Artifacts `goal-pursuit-gtn-230m{-dense,-dense-control}.*`.

A correction to the serving number, from the same law. Section 5 reports the trained errand policy reaching 0.40 through the real serving path against a 1.00 harness ceiling, and calls the 0.60 shortfall the serve wrapper’s cost. That attribution was wrong, and the law this note is about says why. The isolated instance armed one block — the self block — and never fed it; an unfed block’s features are `None`, the server’s composition then applies no injection at all, and so the policy bridge, trained under a *dream4-armed* host, was served against a *bare* one. Retrained under the bare host by the identical recipe, seed, and chains, the same policy scores **1.00** through the same serving path — the same wrapper, the same cached-generate delivery, the same prompt template (which, measured directly, is byte-identical to the harness’s) — with the zero-shot serve floors still at 0.00 and the no-goal-text control at 0.30. The serve wrapper costs nothing. The transfer gap was the training host, one more time, and the delivery path is clean end to end. Artifacts `goal-pursuit-g3b-230m.*` join the bundle; the original G3 run stands as the mismatched half of the pair.

regime bridge trained under	none	countdown	temporal block
dream4-only (original)	1.00/1.00/1.00	1.00/1.00/1.00	0.00/0.00/0.00
dream4+temporal (re-run)	0.00/0.00/0.00	0.00/0.00/0.00	1.00/1.00/1.00

Table 3: The pair of GT-230M runs, per seed: train/serve host match fully determines the outcome, in both directions.

References

- [1] F. Penov. The Carrier Holds, the Emitter Leaks. Preprint, 2026. <https://github.com/kortexa-ai/legolm.basis>.
- [2] F. Penov. The Writer Wins, the Emitter Learns. Preprint, 2026. <https://github.com/kortexa-ai/legolm.basis>.
- [3] F. Penov. The Third Block Composes. Preprint, 2026. <https://github.com/kortexa-ai/legolm.basis>.
- [4] F. Penov. Zero Weight-Space Interference Is Not Enough. Preprint, 2026. <https://github.com/kortexa-ai/legolm.basis>.
- [5] Liquid AI. LFM2.5 model family. Model releases, 2025–2026. <https://huggingface.co/LiquidAI>.
- [6] Qwen Team. Qwen3.5 and Qwen3.6 model families. Model releases, 2025–2026. <https://huggingface.co/Qwen>.